Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



Team Details

a. Team name: **Royal Stack**
b. Team leader name: **SATHISHKUMAR S**
c. Problem Statement: Retail & Commerce. Small and mid-size Indian retailers run on billing software: shoppers get no help in the aisle, owners guess their reorders, and nothing is connected. Royal Stack is one AI commerce agent on one inventory truth, serving the store visit, the online shop and the owner's dashboard.

Brief about the idea

Royal Stack: one AI commerce agent, one inventory truth.

One Gemini agent team serves the shopper in the store and online and runs the owner's inventory on one stock record, filled from the software the shop already runs. Two shopper experiences share one session, cart and checkout: the store visit (entrance QR, voice assistant with map and aisle directions, scan to add, exit kiosk) and the online shop (product-first home, product sheet with label facts and price comparison, cart, checkout). The owner runs forecasts, reorders, demand signals, customers and integrations on one dashboard.

In the aisle

The gap: No one to ask, no answer in the shopper's language, a queue at the end.

What we built: The store visit: entrance QR, no install; voice assistant in 25 languages with map and aisle directions; scan to add; exit kiosk with seven verification layers.

Online

The gap: A small shop's online store, when it has one, does not know the shelf stock and has no assistant.

What we built: The online shop on the same session, cart and checkout: categories, deals, trending from real orders, picks for signed-in shoppers, label facts with citations, price comparison across providers.

In the back office

The gap: Reorders from memory, demand noticed after the sale is lost, stock data locked inside billing software.

What we built: Owner dashboard with BigQuery ML forecasts, reorder drafts with a Gemini rationale, demand signals and customer segments; a connector agent that brings files, sheets, API feeds, databases and MCP servers in.

Everything on these slides is deployed at royastacxk-aibuildercup.web.app on Google Cloud with Gemini. Nothing here is planned-only.

Opportunities

How it differs from existing ideas

- Most retail AI is a chatbot on one channel. Royal Stack is an agent team on Google ADK (a coordinator plus specialists on Gemini) that serves the store visit, the online shop and the owner on one inventory truth.
- Real-time speech to speech on Gemini Live in 25 languages; typed lines join the same spoken conversation; every fact comes from a tool over the live catalog.
- The shop's catalog is itself an MCP server (/mcp/stores/{store_id}), and the shop can connect any MCP provider to compare prices.

How it solves the problem

- Shoppers get stock, price, aisle directions and label facts with citations from tools, never guessed; out-of-stock items come with alternatives.
- Owners get ARIMA_PLUS forecasts, days of cover, reorder drafts grouped by supplier with a rationale, demand signals and customer segments; nothing changes without approval.
- Inventory comes from what the shop already has: Excel template, Google Sheets copy, checked upload, scan, API feeds, databases, MCP servers. One taxonomy chain is applied at every import, so search, the map and spoken directions agree.

USP

- The model talks, code decides: prices, stock, discounts, points and exit verdicts are code inside transactions; Gemini handles language, vision, grounding and proposals.
- One session, cart and checkout across the store visit and the online shop.
- Self-service: a shop registers, uploads its inventory and is live. 380 backend and 32 frontend tests; every control mapped in docs/security.md.

Judging criteria addressed here: innovation (25%) and problem alignment (25%); technical merit and GenAI use (40%) on the architecture and technology slides; UX (10%) on the screens.

List of features offered by the solution

Shopper: the store visit

- Entrance QR, no app install; optional phone OTP sign-in for history, offers and points
- Voice assistant on Gemini Live: real-time speech to speech, interruptible, 25 languages; typed lines join the conversation
- Floor map with aisle, bay and side directions; aisle QRs update where you stand
- Scan to add; offers apply themselves; points at checkout; sandbox payment; receipt QR
- Exit kiosk: seven verification layers decide; Gemini vision only moves confidence

Shopper: the online shop

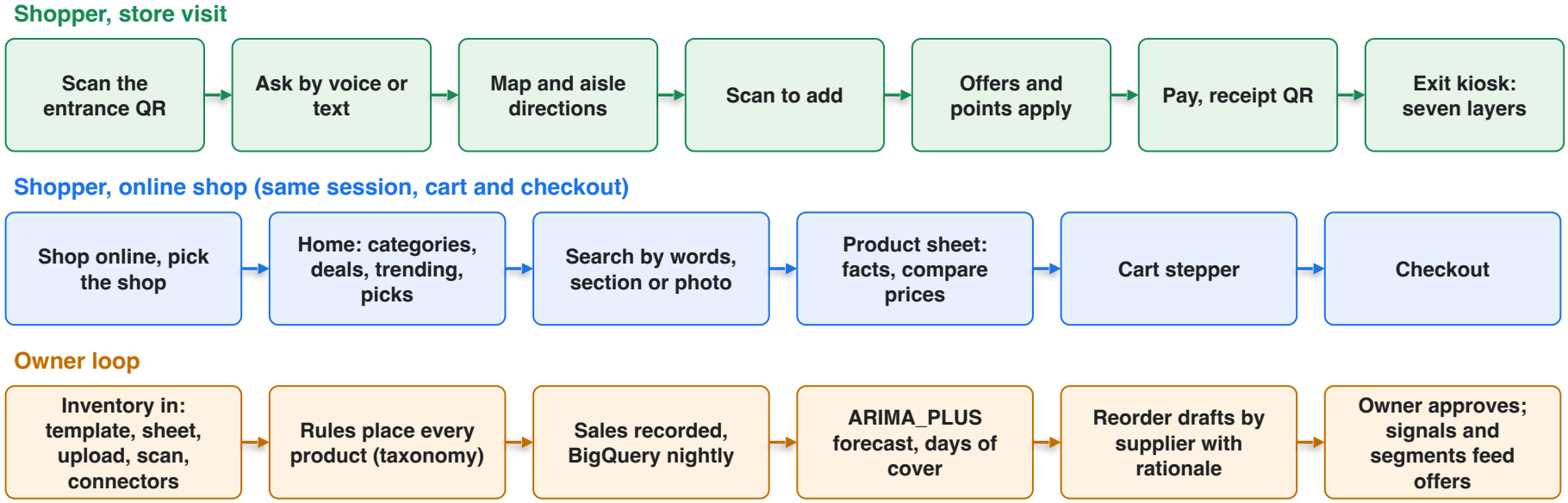
- Product-first home: categories, deals, trending from real orders, picks for signed-in shoppers
- Search by words, by section ("healthy food" lists that section) or by photo
- Product sheet: quantity stepper, label facts from public sources with citations, price comparison across providers
- Cart stepper; the same session, cart and checkout as the store visit
- Assistant tools over the live catalog: search, section listing, store overview, navigation, cart, offers, product facts, compare prices

Owner dashboard

- Today's sales; inventory with BigQuery ML ARIMA_PLUS forecasts and days of cover; product editor
- Reorder drafts grouped by supplier with a Gemini rationale; approve or reject
- Demand signals; customer segments (new, regular, VIP, at risk); offers; loss and shrink; exit checks; account settings
- Inventory intake: Excel template, Google Sheets copy, checked upload (no accidental overwrite), add by hand or by scan
- Integrations: connector agent for files, sheets, API feeds, databases and MCP servers (propose mapping, preview, confirm; credentials in Secret Manager); the catalog served as an MCP server

Taxonomy: product → category → subcategory → storage → department → floor block (Staples & Cooking; Snacks, Beverages & Sweets; Personal Care & Baby; Household, Pooja & Pets; Home, Kitchen & Hardware; Stationery, Toys & Sports; Fresh Produce; Clothing & Footwear; Chiller; Freezer), applied by rules at every import, shown on the map and spoken in directions.

Process flow diagram or Use-case diagram



How a turn is answered

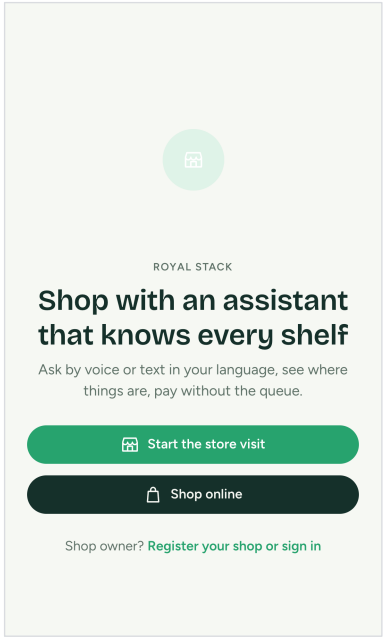
A coordinator with no tools routes each turn to a specialist (catalog, navigation, cart; for owners: briefing with two scouts in parallel and a writer, inventory planner, customers and offers, data connector, store ops). Specialists act only through tools over the live catalog; the screen renders tool results.

Rules decide at the exit

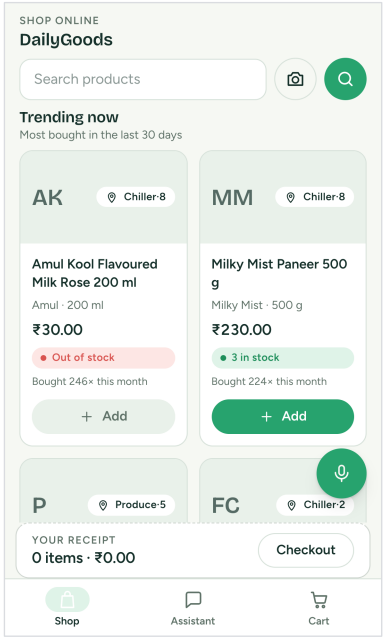
Receipt token, paid order, item count, line limit, high-value items, random audit, then a Gemini bag photo that only moves confidence and never blocks an exit. Staff review flags with a kiosk PIN; photos are never stored.

Wireframes/Mock diagrams of the proposed solution (optional)

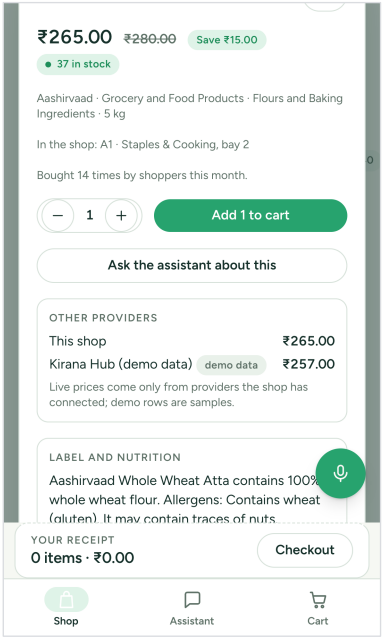
Live screens of the deployed app on a phone, captured 18 Oct 2026 (no mock-ups).



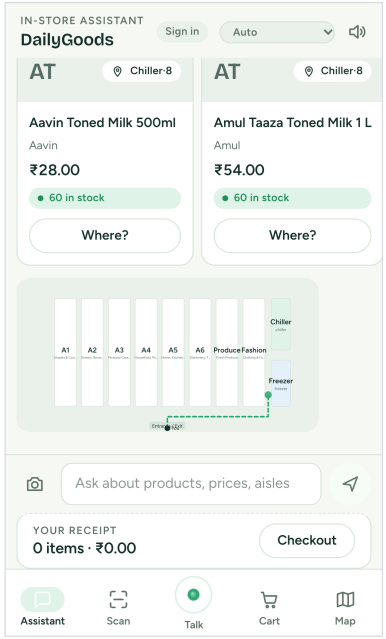
Landing: two front doors, the store visit and the online shop



Online shop: categories, deals, trending from real orders



Product sheet: price comparison across providers, label facts with citations



Store visit: assistant answers from the catalog and lights the aisle on the map

Architecture diagram of the proposed solution

Clients

Shopper PWA on the phone (store visit and online shop) · Owner dashboard on laptop or phone · Exit kiosk on a tablet · Other agents over MCP. Static Next.js PWA on Firebase Hosting; Firebase Auth: Google and email for owners, phone OTP for shoppers who opt in.

Cloud Run: FastAPI (Python 3.12) with Google ADK 2.12 agent teams

- Shopper coordinator → catalog, navigation, cart specialists; owner coordinator → briefing (two scouts in parallel, one writer), inventory planner, customers and offers, data connector, store ops
- Gemini Live websocket bridge: speech to speech, echo guard, typed lines in the same session, transcripts in the spoken script
- Connector agent and normalizer (files, Sheets, API feeds, databases, MCP); MCP servers for every store's catalog and for the demo provider
- Taxonomy rules at every import; signed sync (HMAC, replay-safe); rate limits; audit on every change

Vertex AI · Gemini

- Gemini 3.5 Flash: tools, vision, mapping, rationale
- Gemini 3.1 Pro: heavier reasoning
- Gemini Live 2.5 Flash: speech to speech
- gemini-embedding-001: multilingual search
- Google Search grounding: product facts with citations

Firestore

Inventory truth, orders, customers, points ledger, memory, audit; deny-all client rules

BigQuery ML

Sales history; ARIMA_PLUS forecasts per product and store, nightly

Cloud Scheduler

Forecast nightly, customers nightly, sync and connectors every 15 min

Secret Manager · Cloud TTS

Signing keys, connector credentials, MCP tokens; Text-to-Speech for the demo narration

Outside the platform

Billing software exports (Excel, CSV, JSON, PDF), Google Sheets, API feeds, Postgres/MySQL and MCP providers through the connector agent · Open Food Facts for barcodes · Razorpay (sandbox) for payment · GitHub Actions CI with the Firestore emulator, Cloud Build source deploys

Technologies to be used in the solution

Google Cloud and AI

- Vertex AI Gemini 3.5 Flash (tools, vision, grounding, rationale) and Gemini 3.1 Pro
- Gemini Live 2.5 Flash (speech to speech)
- gemini-embedding-001 (search); Google Search grounding (product facts)
- Google ADK 2.12: coordinator, specialist, parallel and sequential agents
- Cloud Run, Cloud Build, Cloud Scheduler
- Firestore; BigQuery ML (ARIMA_PLUS)
- Secret Manager; Cloud Text-to-Speech (demo narration)

Firebase and web

- Firebase Hosting (static PWA)
- Firebase Auth: Google, email and password, phone OTP
- Next.js 16, TypeScript, Tailwind v4, zustand
- Web Audio and AudioWorklet for live voice
- Barcode and QR scanning in the browser
- Recharts, qrcode.react
- vitest (32 tests), lint and static build in CI

Backend and data

- Python 3.12, FastAPI, google-genai SDK
- MCP over Streamable HTTP JSON-RPC: hosted servers and a guarded client
- openpyxl (Excel template and upload), SQLAlchemy, psycopg, PyMySQL, httpx
- HMAC-signed sync contract, outbox with retries
- Open Food Facts for real barcodes
- pytest on the Firestore emulator (380 tests), GitHub Actions CI
- Design notes per module in docs/lld, control map in docs/security.md

Why these: Gemini for language, vision, grounding and proposals; Firestore transactions for anything that counts or pays; BigQuery ML for forecasts without an ML pipeline; Cloud Run so one shop costs a warm instance, not a server; MCP so other agents can shop the catalog.

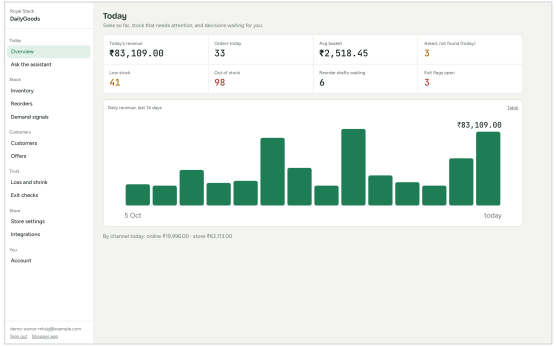
Estimated implementation cost (optional)

Indicative monthly cost for one shop at pilot scale (about 300 visits a day, one warm API instance, asia-south1). List prices change; the usage lines scale with conversations.

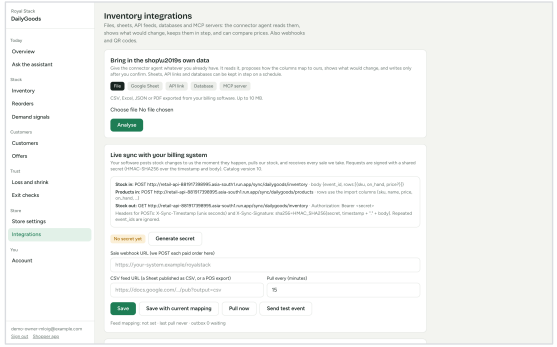
Component	What drives it	Indicative cost per month
Cloud Run API	One warm instance (1 vCPU, 1 GiB), scales to three under load	₹1,200 to ₹2,500
Gemini usage	Voice: about ₹5 per visit of 8 turns. Text: under ₹0.50 per question. Forecast rationale, mapping and product facts: a few paise each	₹4,000 to ₹12,000 depending on voice share
Firestore	Catalog, carts, orders, customers, memory; mostly inside the free tier at this scale	₹0 to ₹500
BigQuery ML	Nightly ARIMA_PLUS retraining per store, small tables	₹0 to ₹300
Firebase Hosting and Auth	Static hosting, Google and email sign-in free; phone OTP per SMS beyond the free allowance	₹0 to ₹500
Scheduler, Secret Manager, Build	Three jobs, a handful of secrets, source deploys	under ₹200
Total	Platform fixed cost plus usage; no per-seat licence, no hardware beyond a phone and an optional tablet	about ₹6,000 to ₹16,000

Snapshots of the prototype

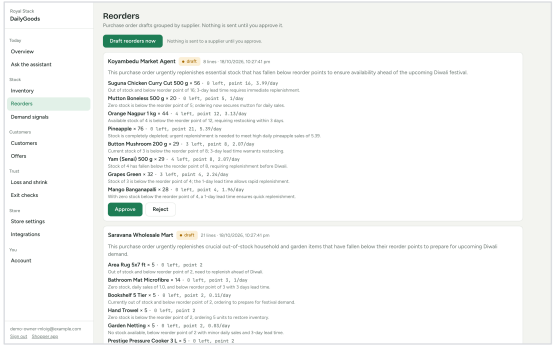
Owner dashboard of the demo store DailyGoods, captured 18 Oct 2026 on a laptop; the phone screens are on the wireframes slide.



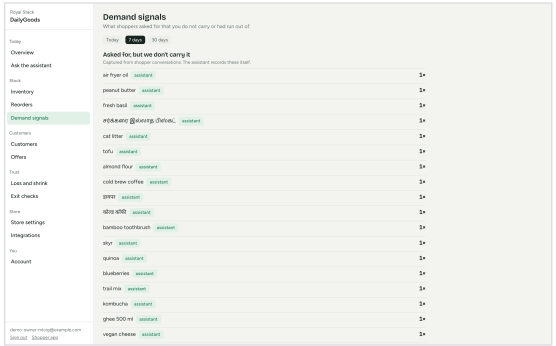
Today: sales so far, low stock, open decisions



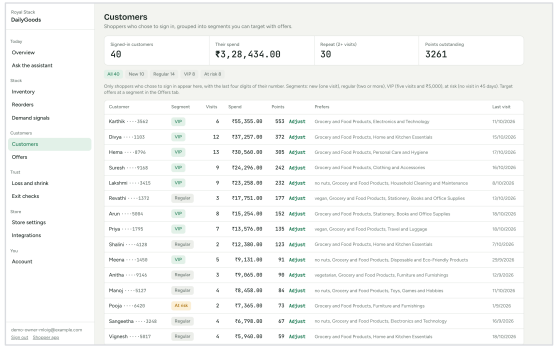
Integrations: file, sheet, API, database, MCP



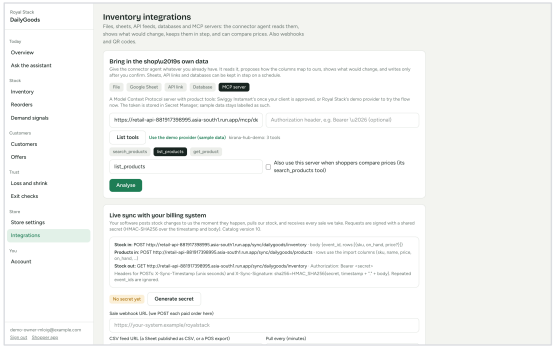
Reorders by supplier with a Gemini rationale



Demand signals: asked for but not carried



Customers: segments, spend, points



MCP source: list tools, preview, confirm

Prototype Performance report/Benchmarking

Measured on the deployed Cloud Run service on 18 Oct 2026 from a laptop in India; latencies are medians of three runs.

Measure	Result
Store catalog read (DailyGoods, 1,541 products)	117 ms
Text question with tool calls: first visible event	1.2 to 2.0 s
Text question: full streamed reply (search, navigation, cart add)	5.9 to 9.3 s
Live voice: first spoken audio after the question	1.8 s
Live voice: a 22-second spoken answer fully delivered	5.0 s (plays while it streams)
Catalog refresh across API instances after a stock change	within 2 s (version counter)
Languages in the selector	25, each tag verified against the live model; transcripts in the spoken script
Automated tests	380 backend (pytest on the Firestore emulator), 32 frontend (vitest); GitHub Actions CI
Live checks on the deployed service	Every feature: online shop, taxonomy, MCP, product facts, typed lines, inventory intake, product editor, account, Tamil voice, echo guard
Security	Every control mapped to its code path and its test in docs/security.md; known gaps listed there

Additional Details/Future Development (if any)

Next 30 days

- Shopify adapter on the existing sync contract (Admin GraphQL, orders and inventory webhooks) and POS file drops from Cloud Storage
- GST invoices and returns against a receipt
- 90-day retention job for verification records and conversation turns
- Phone sign-in for non-Indian numbers, logo upload, a retry button for dead outbox entries

Next quarter

- Named adapters (Zoho Inventory, Tally Prime) on the connector agent
- Real provider MCP servers (Swiggy Instamart once production access is granted) alongside the demo provider
- Home delivery and order tracking for the online shop
- Supplier portal: purchase orders sent and acknowledged

Pilots and the business

- Self-service registration is open today: a shop registers, uploads its inventory on the Excel or Sheets template and is live
- Pricing idea: a flat platform fee per shop plus usage; no hardware beyond a phone and an optional tablet
- Target: neighbourhood supermarkets in Tamil Nadu first, then any JAPAC market the model speaks
- Measure: lost-sale signals recovered, shrink reduced, queue time removed

Known gaps kept on purpose are listed at the end of docs/security.md; the roadmap is in docs/plan/09-roadmap-after-submission.md.

Provide links to your:

- GitHub public repository: https://github.com/DSsathish/royalstack_aibuilderhackathon
- **Demo video: Google Drive link (to be added)**
- Final product link: https://royastacxk-aibuildercup.web.app

API (OpenAPI at /docs): https://retail-api-881917398995.asia-south1.run.app

MCP index (the catalog as an MCP server): https://retail-api-881917398995.asia-south1.run.app/mcp

Owner dashboard: https://royastacxk-aibuildercup.web.app/owner/

Exit kiosk: https://royastacxk-aibuildercup.web.app/kiosk/

How to try it

- Shopper: open the live app on a phone, tap Shop online and pick DailyGoods; or scan the entrance QR from the owner's Integrations tab for the store visit
- Owner: /owner/ registers a shop with a Google or email account in a minute; upload the Excel template or connect a sheet, API, database or MCP server
- Any MCP client can call /mcp/stores/dailygoods (store_info, search_products, get_product, list_category)

In the repository

- README.md: what it does, Google Cloud services, run, test and deploy
- docs/hld.md and docs/lld/*.md: design notes per module, written before the code
- docs/security.md: every control mapped to its code path and test, plus known gaps
- docs/demo-script.md and docs/submission-checklist.md

How Royal Stack maps to the judging criteria

Technical merit and GenAI use (40%)

- Google ADK 2.12 agent teams on Gemini 3.5 Flash: a coordinator and specialists, two scouts in parallel and a writer for the owner briefing
- Gemini Live speech to speech over a Cloud Run websocket bridge, 25 languages, typed lines in the same session
- Google Search grounding with citations for product facts; gemini-embedding-001 search; BigQuery ML ARIMA_PLUS forecasts
- MCP server and client; Firestore transactions for anything that counts or pays; 380 backend and 32 frontend tests; control map in docs/security.md

Problem alignment (25%)

- Retail & Commerce: small and mid-size Indian retailers that already run billing software
- In the aisle: assistant, map and aisle directions, scan to add, exit kiosk
- In the back office: forecasts, reorder drafts with a rationale, demand signals, customer segments
- Between systems: inventory from Excel, Sheets, checked upload, scan, API feeds, databases and MCP servers; the catalog served back as MCP

Innovation (25%)

- One agent team, one inventory truth, two shopper experiences on one session, cart and checkout
- The model talks, code decides: prices, stock, discounts, points and exit verdicts are code
- The shop as an MCP server for other agents; price comparison across MCP providers
- Seven-layer exit where vision only moves confidence; one taxonomy chain applied at import, drawn on the map, spoken in directions

User experience (10%)

- No install: the entrance QR or Shop online; sign-in optional and only the last four digits of a number ever shown
- Voice in 25 languages, interruptible, with full text equivalence and transcripts in the spoken script
- Product-first home, label facts with citations, price comparison, cart stepper, offers that apply themselves
- Owners preview before every import and approve before any reorder is sent; light and dark themes, screen-reader labels, reduced motion respected

Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.

Thank you

